Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота №2

з дисципліни
«Проєктування складних систем»

на тему

«Дослідження наслідків роботи технології апаратного пропуску

замків (Intel’s Hardware Lock Elision)»

Виконав:

студент групи ІМ-21

Сірик Максим

Варіант 20

Перевірив:

Долголенко Олександр

Київ 2026

Мета роботи: Дослідити наслідки роботи технології апаратного пропуску замків (Intel's Hardware Lock Elision)

Постановка завдання

Для виконання цієї роботи потрібен доступ до мікропроцесора Intel, що підтримує технологію TSX-NI, або TSX/TSX-NI Suspend Load Address Tracking (TSXLDTRK, with Sapphire Rapids, 2023).

Початкові дані введіть з файлу, збереженого в першій лабораторній роботі. Створіть потоки з використанням, наприклад, інтерфейсу anonymous Runnable з пакету java.util.concurrent (синхронізація потоків за допомогою інтерфейсу Lock із пакета java.util.concurrent.lock). Дослідіть швидкість виконання Вашої паралельної програми на цьому мікропроцесорі (тільки один варіант програми, коли два, або більше потоків обчислюють паралельно обидві функції варіанту). Після цього відключіть Intel® TSX-NI за допомогою наступного налагоджування реєстру (для Windows 10): reg add "HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\Session Manager\Kernel" /v DisableTsx /t REG_DWORD /d 1 /f. Перегрузіть комп'ютер, щоб ці зміни мали ефект та знову дослідіть швидкість виконання Вашої паралельної програми. Побудуйте порівняльні графіки й зробіть висновки по роботі.

Ваш варіант математичних функцій для обчислення двох перших лабораторних робіт визначається Вашим номером в списку групи (20) (де: MA і т. п. – матриця, розміром nxn; A і т. п. – вектор, розміром n; a і т. п. – скаляр):

ВАРІАНТ 20

MG = MB*MT + MC*(MB*MM+MT);

B = MT*D - min(D)*C.

Виконання завдання

Згідно з openjdk-17 (і також 22, яку я використовую), потрібно вмикнути flag-и `-XX:+UnlockExperimentalVMOptions -XX:+UseRTMLocking`.

1. https://manpages.debian.org/bullseye/openjdk-17-jre-headless/java.1.en.html#XX:+UseRTMLocking
2. https://docs.oracle.com/en/java/javase/22/docs/specs/man/java.html

> *Generates Restricted Transactional Memory (RTM) locking code for all inflated locks, with the normal locking mechanism as the fallback handler. This option is* ***disabled by default****. Options related to RTM are available only on x86 CPUs that support Transactional Synchronization Extensions (TSX).*

Графіки

ПСС_ЛР2_ІМ-21_Сірик_Максим_Олександрович

N	500	550	600	650	700	750	800	850	900	950	1000
Час виконання без замків (ms)	1378	1415	1920	2179	3530	3782	5869	7414	8787	9998	13737
	957	1186	1619	2136	3364	3934	5830	7438	8931	10124	13934
	904	1708	1920	2769	3448	4567	6016	7284	9180	10798	14941
	1053	1550	1851	2688	3749	4528	6263	7750	9215	11388	13340
	1130	1313	2295	2502	3482	4501	5732	7621	9244	11048	15193
	985	1499	1904	2819	3611	4505	5997	7643	9198	11675	13514
Average (ms)	1067.833333	1445.166667	1918.166667	2515.5	3530.666667	4302.833333	5951.166667	7525	9092.5	10838.5	14109.83333
N	500	550	600	650	700	750	800	850	900	950	1000
Час виконання із замками	2873527477	2915509543	4234588005	7451072337	8569444140	1096412051 0	1239919363 8	1441837315 9	2043984400 9	2109776478 2	2806729778 9

TSX off (ns)											
	2429890592	5242854270	4377378080	5073871504	7963266332	9866455191	1308924364 3	1312322443 4	1602839250 9	1948509563 8	2829491438 9
Average (ms)	2651.709035	4079.181907	4305.983043	6262.471921	8266.355236	10415.28785	12744.21864	13770.7988	18234.11826	20291.43021	28181.10609
N	500	550	600	650	700	750	800	850	900	950	1000
Час виконяння із замками TSX on (ms)	1158626258	1576844846	2134666844	2767667940	3874495249	5001987601	8493368662	8108720593	9917080786	1200923819 1	1605293105 9
	1246428534	1686604310	2186279190	3821587776	4155428711	6167514506	8702443933	8553201411	11826950102	1308853363 0	2256010358 7
Average (ms)	1202.527396	1631.724578	2160.473017	3294.627858	4014.96198	5584.751054	8597.906298	8330.961002	10872.01544	12548.88591	19306.51732

Лістинг

```
import java.io.File;
import java.io.FileNotFoundException;
import java.io.FileWriter;
import java.io.IOException;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Random;
import java.util.Scanner;
import java.util.concurrent.BrokenBarrierException;
import java.util.concurrent.CyclicBarrier;
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;
import java.util.function.Function;
import java.util.function.Supplier;

interface IMatrix {
    static Matrix multiply(IMatrix MA, IMatrix MB, int rowStart, int
rowEnd, int colStart, int colEnd, Lock lock) {
        if (rowEnd == -1) rowEnd = MA.size(0) - 1;
        if (colEnd == -1) colEnd = MB.size(1) - 1;

        int m = MA.size(0);
        int n = MA.size(1);
        int p = MA.size(1);

        assert n == MB.size(0) : "Can't multiply matrices";
        assert rowEnd >= rowStart : "rowEnd must be greater than or
equal to rowStart";
        assert rowEnd < m : "rowEnd must be less than the number of rows
in MA";
        assert colEnd >= colStart : "colEnd must be greater than or
equal to colStart";
        assert colEnd < p : "colEnd must be less than the number of
columns in MB";

        int numRows = rowEnd - rowStart + 1;
        int numCols = colEnd - colStart + 1;
        double[][] result = new double[numRows][numCols];

        for (int i = rowStart; i <= rowEnd; i++) {
            for (int k = colStart; k <= colEnd; k++) {
                final int ii = i;
                final int kk = k;
                lock.lock();
```

```
                var sum = Summator.kahanSum(0, n - 1, (j) -> MA.get(ii,
j) * MB.get(j, kk));
                result[i - rowStart][k - colStart] = sum;
                lock.unlock();
// Equivalent to:
//                for (int j = 0; j < n; j++) {
//                    result[i - rowStart][k - colStart] += MA.get(i, j)
* MB.get(j, k);
//                }
            }
        }

        return new Matrix(result);
    }

    static Matrix multiplyScalar(IMatrix MA, double scalar, int
rowStart, int rowEnd, int colStart, int colEnd, Lock lock) {
        if (rowEnd == -1) rowEnd = MA.size(0) - 1;
        if (colEnd == -1) colEnd = MA.size(1) - 1;

        assert rowEnd >= rowStart : "rowEnd must be greater than or
equal to rowStart";
        assert rowEnd < MA.size(0) : "rowEnd must be less than the
number of rows in MA";
        assert colEnd >= colStart : "colEnd must be greater than or
equal to colStart";
        assert colEnd < MA.size(1) : "colEnd must be less than the
number of columns in MA";

        int m = rowEnd - rowStart + 1;
        int n = colEnd - colStart + 1;

        double[][] result = new double[m][n];

        for (int i = rowStart; i <= rowEnd; i++) {
            for (int j = colStart; j <= colEnd; j++) {
                lock.lock();
                result[i - rowStart][j - colStart] = MA.get(i, j) *
scalar;
                lock.unlock();
            }
        }

        return new Matrix(result);
    }
```

```
    static Matrix sum(IMatrix MA, IMatrix MB, Lock lock) {
        return sum(MA, MB, 1, lock);
    }

    static Matrix diff(IMatrix MA, IMatrix MB, Lock lock) {
        return sum(MA, MB, -1, lock);
    }

    private static Matrix sum(IMatrix MA, IMatrix MB, int sign, Lock
lock) {
        sign = Integer.signum(sign);

        int m = MA.size(0);
        int n = MA.size(1);

        assert m == MB.size(0) : "Can't sum matrices";
        assert n == MB.size(1) : "Can't sum matrices";

        double[][] result = new double[m][n];

        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                double[] values = {MA.get(i, j), sign * MB.get(i, j)};
                lock.lock();
                result[i][j] = Arrays.stream(values).sorted().sum();
                lock.unlock();
            }
        }

        return new Matrix(result);
    }

    static double min(IMatrix A, int rowStart, int rowEnd, int colStart,
int colEnd, Lock lock) {
        if (rowEnd == -1) rowEnd = A.size(0) - 1;
        if (colEnd == -1) colEnd = A.size(1) - 1;

        assert rowEnd >= rowStart : "rowEnd must be greater than or
equal to rowStart";
        assert rowEnd < A.size(0) : "rowEnd must be less than the number
of rows in A";
        assert colEnd >= colStart : "colEnd must be greater than or
equal to colStart";
        assert colEnd < A.size(1) : "colEnd must be less than the number
of columns in A";
```

```java
        double minValue = Double.MAX_VALUE;

        for (int i = rowStart; i <= rowEnd; i++) {
            for (int j = colStart; j <= colEnd; j++) {
                double value = A.get(i, j);
                lock.lock();
                minValue = Math.min(minValue, value);
                lock.unlock();
            }
        }

        return minValue;
    }

    double get(int i, int j);

    int size(int dim);

    class TransposeView implements IMatrix {
        private final IMatrix MA;

        public TransposeView(IMatrix MA) {
            this.MA = MA;
        }

        @Override
        public double get(int i, int j) {
            return MA.get(j, i);
        }

        @Override
        public int size(int dim) {
            if (dim == 0) return MA.size(1);
            else if (dim == 1) return MA.size(0);
            else
                throw new IllegalArgumentException("Invalid dimension: "
+ dim);
        }
    }

    class OffsetView implements IMatrix {
        private final IMatrix MA;
        private final int colOffset;
        private final int colLen;

        public OffsetView(IMatrix MA, int colOffset, int colLen) {
```

```
            this.MA = MA;
            this.colOffset = colOffset;
            this.colLen = colLen;
        }

        @Override
        public double get(int i, int j) {
            return MA.get(i, j + colOffset);
        }

        @Override
        public int size(int dim) {
            if (dim == 0) return MA.size(0);
            else if (dim == 1) return this.colLen;
            else
                throw new IllegalArgumentException("Invalid dimension: "
+ dim);
        }
    }
}

class Summator {
    static double kahanSum(int start, int end, Function<Integer, Double>
doubleProvider) {
        double sum = 0;
        double err = 0;
        for (int i = start; i <= end; i++) {
            var corrected = doubleProvider.apply(i) - err;
            var newSum = sum + corrected;
            err = (newSum - sum) - corrected;
            sum = newSum;
        }
        return sum;
    }
}

class Matrix implements IMatrix {
    private final double[][] MA;

    public Matrix(double[][] MA) {
        this.MA = MA;
    }

    static Matrix fromVector(double[] vector) {
        var matrix = new double[1][vector.length];
        matrix[0] = vector;
```

```
        return new Matrix(matrix);
    }

    public IMatrix getTransposeView() {
        return new TransposeView(this);
    }

    public IMatrix getOffsetView(int start, int end) {
        return new OffsetView(this, start, end - start + 1);
    }

    public Matrix copy() {
        var copy = new double[MA.length][MA[0].length];
        for (int i = 0; i < MA.length; i++) {
            copy[i] = Arrays.copyOf(MA[i], MA[i].length);
        }
        return new Matrix(copy);
    }

    @Override
    public double get(int i, int j) {
        return MA[i][j];
    }

    @Override
    public int size(int dim) {
        if (dim == 0) return MA.length;
        else if (dim == 1) return MA[0].length;
        else throw new IllegalArgumentException("Invalid dimension: " +
dim);
    }
}

record Data(Matrix MB, Matrix MT, Matrix MC, Matrix MM, Matrix D, Matrix
C) {
    Data deepCopy() {
        return new Data(MB.copy(), MT.copy(), MC.copy(), MM.copy(),
D.copy(), C.copy());
    }
}

record Conf(int N, int amountOfThreads) {
    public Conf {
        assert N % amountOfThreads == 0 : "N must be divisible by
amountOfThreads";
    }
```

```
}

record Triple<F, S, T>(F first, S second, T third) {
    F getFirst() {
        return first;
    }

    S getSecond() {
        return second;
    }

    T getThird() {
        return third;
    }
}

class FileUtils {
    static String baseDir = "data/";

    static String getInputFileName(int N) {
        return N + "_input.txt";
    }

    static String getOutputFileName(int N) {
        return N + "_output.txt";
    }

    public static void writeResult(int N, Triple<Long, Matrix[],
Matrix[]> result) {
        long time = result.getFirst();
        Matrix[] MG_i = result.getSecond();
        Matrix[] B_i = result.getThird();
        var file = new File(baseDir, getOutputFileName(N));
        try (var fileWriter = new FileWriter(file)) {
            fileWriter.write("Time: " + time + " ns\n");
            fileWriter.write("\nMG:\n");
            writeMatrixColParts(fileWriter, MG_i);
            fileWriter.write("\nB:\n");
            writeMatrixRowParts(fileWriter, B_i);
        } catch (IOException e) {
            throw new RuntimeException(e);
        }
    }

    private static void writeMatrixColParts(FileWriter fileWriter,
Matrix[] matrices) throws IOException {
```

```java
        for (int k = 0; k < matrices[0].size(0); k++) {
            for (Matrix matrix : matrices) {
                for (int j = 0; j < matrix.size(1); j++) {
                    var result = matrix.get(k, j) + " ";
                    fileWriter.write(result);
                }
            }
            fileWriter.write("\n");
        }
    }

    private static void writeMatrixRowParts(FileWriter fileWriter,
Matrix[] matrices) throws IOException {
        for (Matrix matrix : matrices) {
            for (int k = 0; k < matrices[0].size(0); k++) {
                for (int j = 0; j < matrix.size(1); j++) {
                    var result = matrix.get(k, j) + " ";
                    fileWriter.write(result);
                }
                fileWriter.write("\n");
            }
        }
    }

    public static Data readData(int N) {
        var file = new File(baseDir, getInputFileName(N));
        if (!file.exists()) {
            generateRandomData(N, file);
        }

        try (var scanner = new Scanner(file)) {
            double[][] MB = new double[N][N];
            double[][] MT = new double[N][N];
            double[][] MC = new double[N][N];
            double[][] MM = new double[N][N];
            double[] C = new double[N];
            double[] D = new double[N];

            for (var matrix : new double[][][]{MB, MT, MC, MM}) {
                for (int i = 0; i < N; i++) {
                    for (int j = 0; j < N; j++) {
                        matrix[i][j] = scanner.nextDouble();
                    }
                }
            }
```

```
            for (var vector : new double[][]{C, D}) {
                for (int i = 0; i < N; i++) {
                    vector[i] = scanner.nextDouble();
                }
            }

            return new Data(new Matrix(MB), new Matrix(MT), new
Matrix(MC), new Matrix(MM), Matrix.fromVector(D), Matrix.fromVector(C));
        } catch (FileNotFoundException e) {
            throw new RuntimeException(e);
        }
    }

    private static void generateRandomData(int N, File file) {
        var random = new Random(42);
        var getDigit = (Supplier<Double>) () -> {
            int digits = random.nextInt(10) + 1;
            double result = Math.round(random.nextDouble() *
Math.pow(10, digits)) / Math.pow(10, digits);
            return result;
        };

        try (var fileWriter = new FileWriter(file)) {
            for (int t = 0; t < 4; t++) {
                for (int i = 0; i < N; i++) {
                    for (int j = 0; j < N; j++) {
                        var num = getDigit.get();
                        fileWriter.write(num + " ");
                    }
                    fileWriter.write("\n");
                }
                fileWriter.write("\n");
            }

            for (int t = 0; t < 2; t++) {
                for (int i = 0; i < N; i++) {
                    var num = getDigit.get();
                    fileWriter.write(num + "\n");
                }
                fileWriter.write("\n");
            }
        } catch (IOException e) {
            throw new RuntimeException(e);
        }
    }
}
```

```
/**
 *
=============================================================================
=====
 *
=============================================================================
=====
 *
=============================================================================
=====
 */
public class Main {
    public static void main(String[] args) throws InterruptedException {
        var threadsCount = 8;
        for (int i = 0; i < 5; i++) {
            var times = new ArrayList<Long>();
            for (int N = 500; N <= 1000; N += 50) {
                var conf = new Conf(N, threadsCount);
                var data = FileUtils.readData(conf.N());
                var result = Runner.run(conf, data, true);
                FileUtils.writeResult(conf.N(), result);
                System.out.println("N: " + conf.N() + ", time: " +
result.getFirst() + " ns");
                times.add(result.getFirst());
            }
            System.out.println(times);
        }
    }
}


class Runner {

    public static Triple<Long, Matrix[], Matrix[]> run(Conf conf, Data
data, boolean isUsingGlobals) throws InterruptedException {
        var ts = new T[conf.amountOfThreads()];
        var threads = new Thread[conf.amountOfThreads()];
        var context = new Context(conf.amountOfThreads(), conf.N());
        for (int i = 0; i < conf.amountOfThreads(); i++) {
            int diapason = conf.N() / conf.amountOfThreads();
            int rowStart = i * diapason;
            int rowEnd = (i + 1) * diapason - 1;
            Data fullCopy = isUsingGlobals ? data : data.deepCopy();
            ts[i] = new T(fullCopy, rowStart, rowEnd, context);
            threads[i] = new Thread(ts[i]);
```

```
        }

        long startTime = System.nanoTime();

        for (var thread : threads) {
            thread.start();
        }

        for (var thread : threads) {
            thread.join();
        }

        long endTime = System.nanoTime();
        long elapsedTime = endTime - startTime;

        Matrix[] result1 = new Matrix[conf.amountOfThreads()];
        Matrix[] result2 = new Matrix[conf.amountOfThreads()];
        for (int i = 0; i < ts.length; i++) {
            result1[i] = ts[i].MG_i;
            result2[i] = ts[i].B_i;
        }

        return new Triple<>(elapsedTime, result1, result2);
    }

    private static class T implements Runnable {
        private final int start;
        private final int end;
        private final Data data;
        private final Context context;
        Matrix MG_i;
        Matrix B_i;

        public T(Data data, int start, int end, Context context) {
            this.data = data;
            this.start = start;
            this.end = end;
            this.context = context;
        }

        private void calc1() {
            var MB_MM_i = IMatrix.multiply(data.MB(), data.MM(), 0, -1,
start, end, context.lock);
            var MB_MM_MT_i = IMatrix.sum(MB_MM_i, data.MT()
                    .getOffsetView(start, end), context.lock);
            var MC_MB_MM_MT_i = IMatrix.multiply(data.MC(), MB_MM_MT_i,
```

```
0, -1, 0, -1, context.lock);
            var MB_MT_i = IMatrix.multiply(data.MB(), data.MT(), 0, -1,
start, end, context.lock);
            MG_i = IMatrix.sum(MB_MT_i, MC_MB_MM_MT_i, context.lock);
        }

        private void calc2() throws BrokenBarrierException,
InterruptedException {
            var min_D_i = IMatrix.min(data.D(), 0, -1, start, end,
context.lock);
            var min_D = context.compareAndGet(min_D_i);
            var min_d_C = IMatrix.multiplyScalar(data.C(), min_D, 0, -1,
start, end, context.lock);
            var MT_D_i = IMatrix.multiply(data.MT(), data.D()
                    .getTransposeView(), start, end, 0, -1,
context.lock);
            B_i = IMatrix.diff(MT_D_i, min_d_C.getTransposeView(),
context.lock);
        }

        @Override
        public void run() {
            try {
                calc1();
                calc2();
            } catch (Exception e) {
                throw new RuntimeException(e);
            }
        }
    }
}

class Context {
    public final Lock lock;
    private final CyclicBarrier minCb;
    private double min = Double.MAX_VALUE;

    public Context(int amountOfThreads, int N) {
        minCb = new CyclicBarrier(amountOfThreads);
        lock = new ReentrantLock();
    }

    public double compareAndGet(double min) throws
BrokenBarrierException, InterruptedException {
        synchronized (this) {
            this.min = Math.min(this.min, min);
```

```
        }
        minCb.await();
        return this.min;
    }
}
```

Висновок

У ході виконання лабораторної роботи було досліджено вплив технології апаратного пропуску замків Intel Hardware Lock Elision на швидкодію паралельної програми. Для цього було розглянуто три режими виконання: без використання замків, із замками при вимкненому TSX та із замками при увімкненому TSX.

За отриманими результатами видно, що найменший час виконання демонструє варіант без замків, оскільки в ньому відсутні накладні витрати на синхронізацію. Варіант із замками при вимкненому TSX показав найгірший результат: зі збільшенням розміру матриць час виконання зростав найбільше, що пояснюється витратами на операції блокування та розблокування, навіть попри те, що в цій реалізації замки фактично не створювали міжпотокового конфлікту, оскільки кожен потік працював зі своїм власним lock-об’єктом.

При увімкненому TSX час виконання програми з замками помітно зменшився порівняно з режимом TSX off і став значно ближчим до варіанту без замків. Це свідчить про те, що апаратне пропускання замків справді дозволяє зменшити накладні витрати синхронізації. Водночас повністю досягти продуктивності варіанту без замків не вдалося, оскільки технологія TSX не усуває всі витрати, а лише намагається оптимізувати виконання критичних секцій, залишаючи звичайний механізм блокування як резервний варіант.

Отже, у межах цієї роботи підтверджено, що використання Intel TSX може суттєво покращити продуктивність паралельної програми у порівнянні зі звичайною реалізацією замків. Разом із тим отримані результати показують, що така оптимізація є залежною від архітектурної підтримки процесора, характеру критичних секцій та особливостей доступу до пам’яті. Тому Intel Hardware Lock Elision слід розглядати як корисну, але не універсальну оптимізацію, ефективність якої визначається конкретними умовами виконання програми.